Deep Learning in Rough Volatility Models

Akmal Abu Bakar, Federico La Penna, Aidan McDougall, Benjamin Kolicic

Department of Mathematics, London School of Economics and Political Science, London, UK

Abstract

This project builds on the deep calibration framework for rough volatility models proposed by Horvath, Muguruza, and Tomas (2019) [1]. As a first goal, the project shows how rough volatility models can be efficiently simulated in such a way as to overcome the high-variance structure of Monte Carlo estimators. Then, the focus moves to the efficiency of the deep calibration implementation and presents a practical two-step feed-forward neural network (FNN) learning procedure (offline learning and online calibration) that permits the routine to be used efficiently. Finally, the practical case of digital barrier option pricing is also addressed.

1 Introduction

The calibration of stochastic volatility models is one of the central challenges in quantitative finance, as classical models such as Heston and SABR provide fast pricing formulas that are highly suitable for industrial applications.

Rough volatility models represent a fundamentally different class. Motivated by the empirical finding of Gatheral, Jaisson, and Rosenbaum (2018) [3] that the realised volatility of equity indices behaves as a fractional Brownian motion with Hurst parameter $H \approx 0.1$, these models replace the classical Markovian variance dynamics with a Volterra process driven by a singular kernel $(t-s)^{H-1/2}$. This rough structure reproduces the steep short-maturity skew observed in equity markets and provides a more accurate description of market dynamics. However, the non-Markovian nature of the variance process rules out many of the classical fast pricing approaches usually employed. Since each volatility surface to be evaluated requires tens of thousands of simulated paths, the calibration of rough volatility models becomes computationally intractable for industrial use.

Horvath et al. [1] propose a way to overcome the problem through a two-step deep learning approach. In the first step, a feed-forward neural network is trained offline to approximate the pricing map from model parameters to implied volatility surfaces, using Monte Carlo-generated data as training labels, such that, once trained, the network replaces the slow simulator entirely. In the second step, the frozen network serves as a fast direct map to be calibrated online via classical gradient-based optimisation. The result is a calibration procedure that matches the accuracy of full Monte Carlo while operating at millisecond speed.

This project replicates the core findings of Horvath et al. [1] and develops the theory underpinning them. After reviewing the preliminaries on neural networks and rough volatility models, Section 3 formalises the calibration problem and the two-step framework that organises the rest of the report. Section 4 carries out the first step: it prices rough Bergomi financial contracts by Monte Carlo and trains the network that approximates the resulting pricing map. Section 5 carries out the second, calibrating the trained map to market data.

2 Preliminaries

In this section we introduce the relevant theory needed to address the deep calibration of rough volatility models: feed-forward neural networks and the rough volatility models themselves.

2.1 Neural networks

Definition 2.1 (feed-forward neural network)**.** A feed-forward neural network with $L-1$ hidden layers is a function $f : \mathbb{R}^{N_0} \to \mathbb{R}^{N_L}$ of the form

$$f = \phi_L \circ A_L \circ \cdots \circ \phi_1 \circ A_1$$

where for $i \in \{1, ..., L-1\}$, $N_i \geq 1$ are the number of nodes in the i^{th} layer; $\phi_i : \mathbb{R}^{N_i} \to \mathbb{R}^{N_i}$ is the activation function and $A_i : \mathbb{R}^{N_{i-1}} \to \mathbb{R}^{N_i}$ is the affine map composed of weights and a bias. The set of all such functions f is denoted as $\mathcal{NN}_L(N_0, N_1, ..., N_L, \phi_1, ..., \phi_L)$.

Theorem 2.2 (Hornik, Stinchcombe and White)**.** *Let $f \in C^n$, $g : \mathbb{R}^{N_0} \to \mathbb{R}$ and $\mathcal{NN}(N_0, 1, \phi)$ be the set of single-layer neural networks with activation function $\phi : \mathbb{R} \to \mathbb{R}$. Then, if the (non-constant) activation function is $\phi \in C^n$, then $g \in \mathcal{NN}(N_0, 1, \phi)$ arbitrarily approximates f and all its derivatives up to order n.*

The above theorem is the universal approximation theorem for derivatives and provides the crucial theoretical foundation for the neural network that will be constructed. The function to be approximated is the map $F^* \colon \Theta \to \mathbb{R}^d$, which maps the model parameters to the corresponding implied volatilities. Since gradient-based calibration requires an accurate approximation of $\nabla_\theta F^*$, the above theorem requires, at a minimum, an activation function $\phi \in C^1(\mathbb{R})$. Indeed, later on, the network will be constructed with the ELU activation function, $\phi_{\text{ELU}}(x) = \begin{cases} x, & x > 0, \\ \alpha\left(e^x - 1\right), & x \leq 0. \end{cases}$, which belongs to $C^1(\mathbb{R})$ and therefore satisfies the requirement. Consequently, the neural network would approximate both the pricing map and its derivatives $\nabla_\theta \widetilde{F} \approx \nabla_\theta F^*$.

2.2 Rough volatility models

Rough volatility models are a class of non-Markovian stochastic volatility models that incorporate a Volterra kernel into their dynamics, characterising their roughness behaviour. In finance, it is now well established that "volatility is rough" (Gatheral et al. [3]) and that financial assets exhibit volatility paths that differ fundamentally from classical Markovian structures such as the Heston model. The introduction of Volterra kernels gives rise to processes sharing the local regularity of fractional Brownian motion, whose characteristic roughness depends on the Hurst parameter ($H \in (0,1)$), with empirical studies finding ($H \approx 0.1$) for equity volatility.

2.2.1 Rough Bergomi model

Definition 2.3 (Rough Bergomi model)**.** On a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t\geq 0}, \mathbb{P})$ the Rough Bergomi model dynamic is defined by

$$dX_t = -\frac{1}{2}V_t dt + \sqrt{V}_t dB_t, \quad X_0 = 0 \tag{1}$$

$$V_t = \xi_0(t)\mathcal{E}(\sqrt{2H}\nu \int_0^t (t-s)^{H-1/2} dW_s), \quad V_0 = v_0 > 0 \tag{2}$$

Where $H \in (0,1)$ is the Hurst parameter, $\mathcal{E}(\cdot)$ is the stochastic exponential, ξ_0 denotes the forward variance curve, and B, W are two $\rho \in [-1,1]$ correlated Brownian motions. dX_t refers to the log-price SDE and V_t is called the variance process.

The roughness in the model enters through the variance process, which depends directly on the Volterra kernel $\int_0^t (t-s)^{H-1/2} dW_s$.

3 Deep calibration

Having introduced neural networks and rough volatility models, we now formalise the calibration problem they serve and present the two-step deep-calibration framework that organises the remainder of this report.

3.1 The calibration problem

In plain terms, calibration chooses the model parameters so that the model reproduces the observed market as closely as possible. To state this precisely, let $\mathcal{M} := \mathcal{M}(\theta)_{\theta\in\Theta}$ be a model with parameter vector $\theta \in \Theta \subset \mathbb{R}^n$. The model and its parameters together fix the prices of any contracts written on the underlying. Let ζ denote those contracts, for instance vanilla options across a set of strikes and maturities, and let the *pricing map*

$$P : \mathcal{M}(\theta, \zeta) \to \mathbb{R}^m, \qquad m \in \mathbb{N},$$

return their model prices, with $\mathcal{P}^{\mathrm{MKT}}(\zeta) \in \mathbb{R}^m$ the corresponding market quotes. Given a metric $\delta(\cdot,\cdot)$ on $\mathbb{R}^m$, the parameter vector $\hat{\theta}$ solves the *ideal δ-calibration problem* if

$$\hat{\theta} = \operatorname*{argmin}_{\theta\in\Theta} \ \delta\big(P(\mathcal{M}(\theta),\zeta),\, \mathcal{P}^{\mathrm{MKT}}(\zeta)\big)\,, \tag{3}$$

with δ chosen to suit the contract ζ at hand.

For most models, and for *every* rough volatility model, the pricing map P has no closed form and must be replaced by a numerical scheme $\widetilde{P}$. Substituting it into (3) gives the *approximate δ-calibration problem*: $\hat{\theta} \in \Theta$ solves it if

$$\hat{\theta} = \operatorname*{argmin}_{\theta\in\Theta} \ \delta\Big(\widetilde{P}(\mathcal{M}(\theta),\zeta),\, \mathcal{P}^{\mathrm{MKT}}(\zeta)\Big)\,. \tag{4}$$

This is the problem we actually solve.

Replacing P by Monte Carlo carries two costs. The first is *speed*. For rough Bergomi the only available $\widetilde{P}$ is the Monte Carlo pricer of Section 4.1, and calibration is iterative: every optimiser step re-evaluates $\widetilde{P}$, so a slow per-evaluation cost compounds over the whole optimisation and makes (4) intractable for industrial use. The second is *accuracy*. Since $\widetilde{P}$ is itself only a noisy estimate of P, any learned approximation can at best reproduce $\widetilde{P}$ up to its sampling error. The training target is therefore $\widetilde{F} = \widetilde{P} + \mathcal{O}(\epsilon)$: we match the numerical pricer to within its own *Monte Carlo error floor* (Section 4.3) rather than drive the error to zero.

3.2 A two-step approach

To remove the speed bottleneck, the two-step approach decouples the slow pricing from the calibration into two steps, one offline and one online.

(i) Offline learning. Replace the slow numerical pricer $\widetilde{P}$ by a neural network that learns the map $\theta \mapsto$ price surface on a *fixed* grid of contracts $\{\zeta_i\}_{i=1}^m$. We first generate a synthetic dataset of parameter–surface pairs $\{(\theta^{(j)}, P^{(j)})\}_{j=1}^N$ with the Monte Carlo simulator of Section 4.1; the labels $P^{(j)}$ are model-generated, not market data. We then select a network $F(\cdot, w)$ from the hypothesis class by empirical-risk minimisation,

$$\widehat{R}_N(F) = \frac{1}{N}\sum_{j=1}^{N} \big\|F(\theta^{(j)}) - P^{(j)}\big\|_2^2, \qquad \widehat{w} = \operatorname*{argmin}_{w} \widehat{R}_N(F(\cdot,w))\,,$$

giving the trained, deterministic approximation $\widetilde{F} := F(\cdot,\widehat{w})$. The output surface over the strike and maturity grid can be regarded as an image of values, a viewpoint we exploit in Section 4.4.

(ii) Online calibration. With $\widetilde{F}$ frozen, deterministic and differentiable, solve the approximate calibration problem (4) against actual market data,

$$\hat{\theta} = \operatorname*{argmin}_{\theta\in\Theta} \ \delta\Big(\widetilde{F}(\theta),\, \mathcal{P}^{\mathrm{MKT}}\Big)\,,$$

now a fast, deterministic, gradient-based least-squares fit on the learned map. The explicit loss and optimiser choice are specified in Section 5.

Step (i) is a one-off offline cost. After it, each online evaluation of $\widetilde{F}$ is a closed-form differentiable map, which brings calibration into the millisecond regime and gives an orders-of-magnitude speed-up over brute-force Monte Carlo.

3.3 Calibrating on the implied-volatility surface

For *vanilla* options we calibrate on the implied-volatility surface rather than on raw prices, as is standard in academia and industry, because it is a better-scaled and more model-agnostic target. The implied volatility $\sigma_{BS}(k,T)$ is defined by Black–Scholes inversion as the unique root of

$$BS(\sigma_{BS}(k,T), S_0, k, T) = P(k,T),$$

computed with the bisection root-finder of Section 4.2.

With this choice the learned map is exactly the map F^* of Section 2.1,

$$F^* : \theta \mapsto \left\{\sigma_{BS}^{\mathcal{M}(\theta)}(T_i, k_j)\right\}_{i=1,\dots,8;\, j=1,\dots,11} \in \mathbb{R}^{88},$$

the offline targets $P^{(j)}$ become implied-vol surfaces $\sigma^{(j)}$, and the online market target becomes $\Sigma_{BS}^{\mathrm{MKT}}$. Because this map is smooth and differentiable in θ, the online step (ii) can be solved with a gradient-based optimiser, relying on the derivative-approximation result of Section 2.1 ($\nabla_\theta \widetilde{F} \approx \nabla_\theta F^*$).

4 Approximate pricing map

This section carries out step (i): we simulate the rough Bergomi underlying (Section 4.1), price the contracts of interest (Section 4.2), and train and test the network that approximates the pricing map (Sections 4.4–4.5).

4.1 Rough Bergomi simulation

We begin with the simulation of the underlying. The Monte Carlo simulation, which for Markovian structures is easily implementable using traditional Euler-scheme approaches, becomes substantially more difficult for rough structures.

4.1.1 The rough Donsker approximation

A usual procedure in the Monte Carlo theory of Gaussian processes is to approximate a Gaussian process by a sequence of functions matching its first and second moments. Among many other approximation procedures, Horvath et al. [2] implement the Itô variance-matching approximation of the Volterra process $\displaystyle\int_0^t (t-s)^{H-1/2} dW_s$ by following the rough Donsker CLT.

Theorem 4.1 (rDonsker CLT for Gaussian Volterra processes)**.** *Let* $(\mathcal{K}\cdot W)_t$ *be a Gaussian Volterra process of this form* $(\mathcal{K}\cdot W)_t = \int_0^t \underbrace{(t-s)^{H-1/2}}_{\mathcal{K}} dW_s$*. The stochastic integral can then be approximated by a sequence of Brownian increments weighted by a deterministic function* g_k^**, where* g_k^* *is chosen in such a way that* $\mathbb{V}(g_k^*\Delta W_k) = [\Delta_k(\mathcal{K}\cdot W)]_t$*. Then, we have*

$$\sum_{k=1}^{n} g_k^* \left(W_{t_k} - W_{t_{k-1}}\right) \to^d \int_0^t (t-s)^{H-1/2} dW_s, \quad \text{as } n \to \infty$$

in the Skorokhod space $\mathcal{D}([0,T])$*.*

Above, $\Delta_k(\mathcal{K}\cdot W) = \int_{t_{k-1}}^{t_k} \mathcal{K} dW_s$ and $[\cdot]_t$ stands for quadratic covariation. See Pakkanen et al. [4] and Horvath et al. [2] for general CLTs of Brownian semi-stationary processes.

Discretization: We now implement the rDonsker discretization. We discretize $[0,T]$ by n steps, with $t_n = t$ and $dt = t/n$

$$\int_0^{t_n} (t_n - s)^{H-1/2} dW_s = \sum_{k=1}^{n} \int_{t_{k-1}}^{t_k} (t_n - s)^{H-1/2} dW_s$$

By theorem 4.1, we approximate the inner integral by Itô variance-matching

$$[\Delta_k(\mathcal{K}\cdot W)]_t = \mathbb{E}\left[\left(\int_{t_{k-1}}^{t_k} (t_n - s)^{H-1/2} dW_s\right)^2\right] =^{\text{Itô isometry}} \int_{t_{k-1}}^{t_k} (t_n - s)^{2H-1} ds$$

$$\mathbb{V}(g_k^*\Delta B_k) = (g_k^*)^2 dt$$

$$\Rightarrow^{\text{matching}} g_k^* = \sqrt{\frac{1}{dt}\int_{t_{k-1}}^{t_k} (t_n - s)^{2H-1}\, ds}$$

Evaluating the inner integral

$$\int_{t_{k-1}}^{t_k} (t_n - s)^{2H-1} ds = \frac{(t_n - t_{k-1})^{2H} - (t_n - t_k)^{2H}}{2H}$$

and substituting $t_k = k \cdot dt$, we get

$$\frac{(dt)^{2H}(n-k+1)^{2H} - (dt)^{2H}(n-k)^{2H}}{2H}$$

Plugging everything back into the main sum, we retrieve the rDonsker approximation

$$\int_0^t (t-s)^{H-1/2} dW_s \approx \sum_{k=1}^{n} \sqrt{\frac{(n-k+1)^{2H} - (n-k)^{2H}}{2H}} dt^H N(0,1) \tag{5}$$

We implemented the above discretization via `JAX` for GPU parallelization with $n = 200$ steps and 60000 simulated paths. Figure 1 shows two paths for Hurst parameter $H = 0.1$.

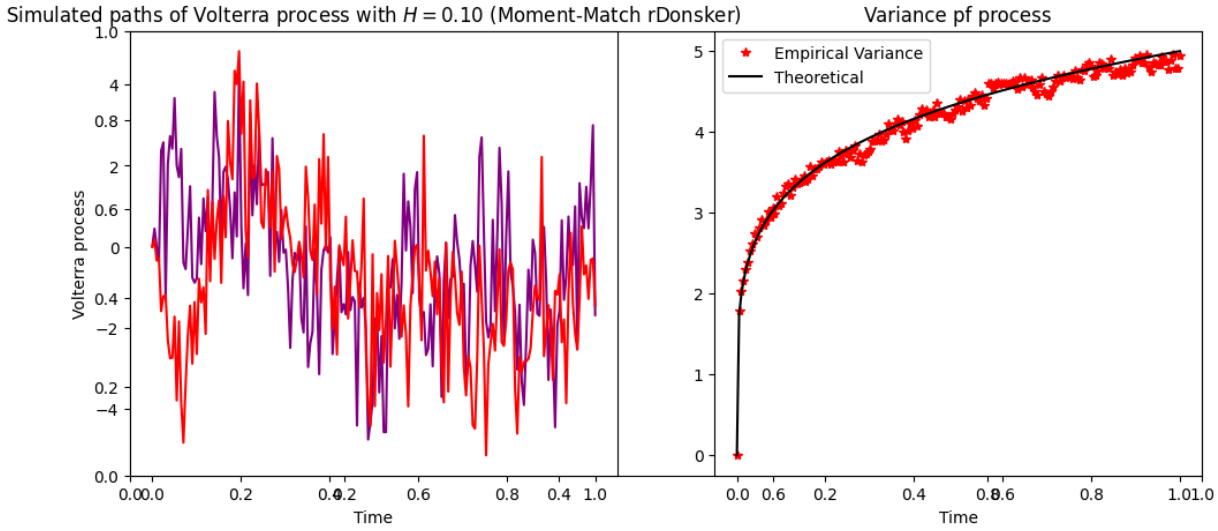

Figure 1: Gaussian Volterra process via rDonsker approximation

4.1.2 Variance and log-price processes

The rough Bergomi model is simulated in two stages. We first evaluate the variance process from the simulated Volterra process, and then simulate the log-price by a classical Euler scheme. The first stage amounts to evaluating the stochastic exponential $\mathcal{E}(\cdot)$ of equation 2. Recall that, the stochastic exponential is defined as $\mathcal{E}(A_t) = \exp\left(A_t - \frac{1}{2}[A]_t\right)$. Let the discretization of $(\mathcal{K} \cdot W)_t$ in Equation 5 be denoted by Y_t. Then

$$\mathcal{E}\left(\sqrt{2H}\nu\,(\mathcal{K}\cdot W)_t\right) = \exp\left(\sqrt{2H}\nu Y_t - \frac{1}{2}\nu^2 t^{2H}\right),$$

where $\left[\sqrt{2H}\nu\,(\mathcal{K}\cdot W)\right]_t = 2H\nu^2\int_0^t (t-s)^{2H-1}ds = \nu^2 t^{2H}$ is its quadratic variation.

Computing V_t explicitly and, by means of the Euler scheme for the log-price with correlated Brownian motion $B_t = \rho W_t + \sqrt{1-\rho^2}\,Z_t$, i.e.

$$X_{t_{k+1}} = X_{t_k} - \frac{1}{2}V_{t_k}dt + \sqrt{V_{t_k}}\,\Delta B_{t_k},$$

we retrieve the model simulation, yielding the variance and log-price paths $\{(V_t^{(i)})_{t\in[0,T]},\ X_T^{(i)}\}_i$ that drive the pricing below.

4.2 Pricing and implied-volatility surfaces

With the underlying paths in hand, we now price the relevant payoffs: vanilla European calls, which we invert to implied volatilities, and down-and-out digital barriers.

4.2.1 Vanilla call

Pricing problem. The price of a European call is the discounted expected payoff $\mathbb{E}\big[(S_0 e^{X_T} - K)^+\big]$, which the standard Monte Carlo estimator approximates by averaging the payoff over the simulated paths. For rough models this estimator suffers from high variance; most visibly, for deep in-the-money strikes it frequently violates the arbitrage bound $C \geq (S_0 - K)^+$. To control this, Horvath et al. [2] apply a variance-reduction scheme that conditions on the variance-driving Brownian motion and prices each path in closed form via Black–Scholes.

Variance reduction. Let $dX_t = -\frac{1}{2}V_t\,dt + \sqrt{V_t}\,dB_t$ and $B_t = \rho W_t + \sqrt{1-\rho^2}Z_t$. We rewrite the SDE as

$$dX_t = -\frac{1}{2}V_t\,dt + \sqrt{V_t}\left(\rho\,dW_t + \sqrt{1-\rho^2}\,dZ_t\right)$$

$$dX_t = \underbrace{\left(-\frac{1}{2}\rho^2 V_t\,dt + \rho\sqrt{V_t}\,dW_t\right)}_{dX_t^{\parallel}} + \underbrace{\left(-\frac{1}{2}(1-\rho^2)V_t\,dt + \sqrt{1-\rho^2}\sqrt{V_t}\,dZ_t\right)}_{dX_t^{\perp}}$$

Integrating over $[0,T]$,

$$X_T^{\parallel} = -\frac{1}{2}\rho^2\int_0^T V_t\,dt + \rho\int_0^T \sqrt{V_t}\,dW_t$$

$$X_T^{\perp} = -\frac{1}{2}(1-\rho^2)\int_0^T V_t\,dt + \sqrt{1-\rho^2}\int_0^T \sqrt{V_t}\,dZ_t$$

$\Rightarrow X_T = X_T^{\parallel} + X_T^{\perp}$. We then condition on $\mathcal{F}_T^W$. $X_T^{\parallel}$ is deterministic, while $X_T^{\perp}|\mathcal{F}_T^W$ is Gaussian, $X_T^{\perp}\,|\,\mathcal{F}_T^W \sim \mathcal{N}\Big(-\frac{1}{2}(1-\rho^2)\int_0^T V_t\,dt,\ (1-\rho^2)\int_0^T V_t\,dt\Big)$.

$$\Rightarrow X_T\,|\,\mathcal{F}_T^W \sim \mathcal{N}\left(X_T^{\parallel} - \frac{1}{2}\underbrace{(1-\rho^2)\int_0^T V_t\,dt}_{\sigma^2},\ \underbrace{(1-\rho^2)\int_0^T V_t\,dt}_{\sigma^2}\right)$$

Therefore $e^{X_T}|\mathcal{F}_T^W$ is log-normal with forward $S_0 e^{X_T^{\parallel}}$ and variance σ^2. The Black–Scholes formula applies: $\mathbb{E}\Big[\max\big(S_0e^{X_T}-K,\,0\big)\,\Big|\,\mathcal{F}_T^W\Big] = BS\Big(S_0e^{X_T^{\parallel}},\,K,\,T,\,\frac{\sigma}{\sqrt{T}}\Big)$. The correspondence between the general solution of Feynman–Kac and this conditional expression is guaranteed by the tower property $\mathbb{E}\big[\max\big(S_0e^{X_T}-K,\,0\big)\big] = \mathbb{E}\Big[\mathbb{E}\Big[\max\big(S_0e^{X_T}-K,\,0\big)\,\Big|\,\mathcal{F}_T^W\Big]\Big]$. In effect, the implementation replaces the standard Monte Carlo payoff averaging with a Black–Scholes evaluation per W path.

Implied-volatility inversion. Once a price has been generated for a strike–maturity pair (K,T) and parameter combination (ξ_0,ν,ρ,H), we recover the implied volatility $\sigma_{\mathrm{BS}}(K,T)$ introduced in Section 3 by inverting the Black–Scholes formula. For this project the inversion is solved with the bisection root-finding method, i.e.

$$\sigma_{\mathrm{BS}} = \mathrm{root}_{\sigma\in[\sigma_{\min},\sigma_{\max}]}\left(C_{\mathrm{BS}}(\sigma) - C_{\mathrm{simul}}\right).$$

The implementation has also been forced to satisfy the classical no-arbitrage bounds for European call options, such as $C \geq (S_0 - K)^+$.

Dataset. The procedure above maps a single parameter vector $\theta = (\xi_0, \nu, \rho, H)$ to a single implied-volatility surface. To learn this map we require many such pairs, so we draw $N = 80000$ parameter vectors by independent uniform sampling over the box

$$\xi_0 \in [0.01, 0.16], \quad \nu \in [0.3, 4], \quad \rho \in [-0.95, -0.1], \quad H \in [0.025, 0.5].$$

The box is restricted to negative ρ and small H to reflect the stylised facts of equity-index volatility, namely the leverage effect and the empirically rough regime $H \approx 0.1$. Each sampled θ is priced on the fixed strike–maturity grid

$$\text{Strikes} = \{0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4, 1.5\},$$
$$\text{Maturities} = \{0.1, 0.3, 0.6, 0.9, 1.2, 1.5, 1.8, 2.0\},$$

yielding an $8 \times 11 = 88$-point surface. The resulting dataset of parameter–surface pairs $\{(\theta^{(j)}, \sigma^{(j)})\}_{j=1}^{80000}$ thus belongs to $\mathbb{R}^{80000\times(4+88)}$ and forms the input–output data on which the FNN is trained and tested. Figure 2 shows four volatility surfaces corresponding to four different parameter combinations in the dataset.

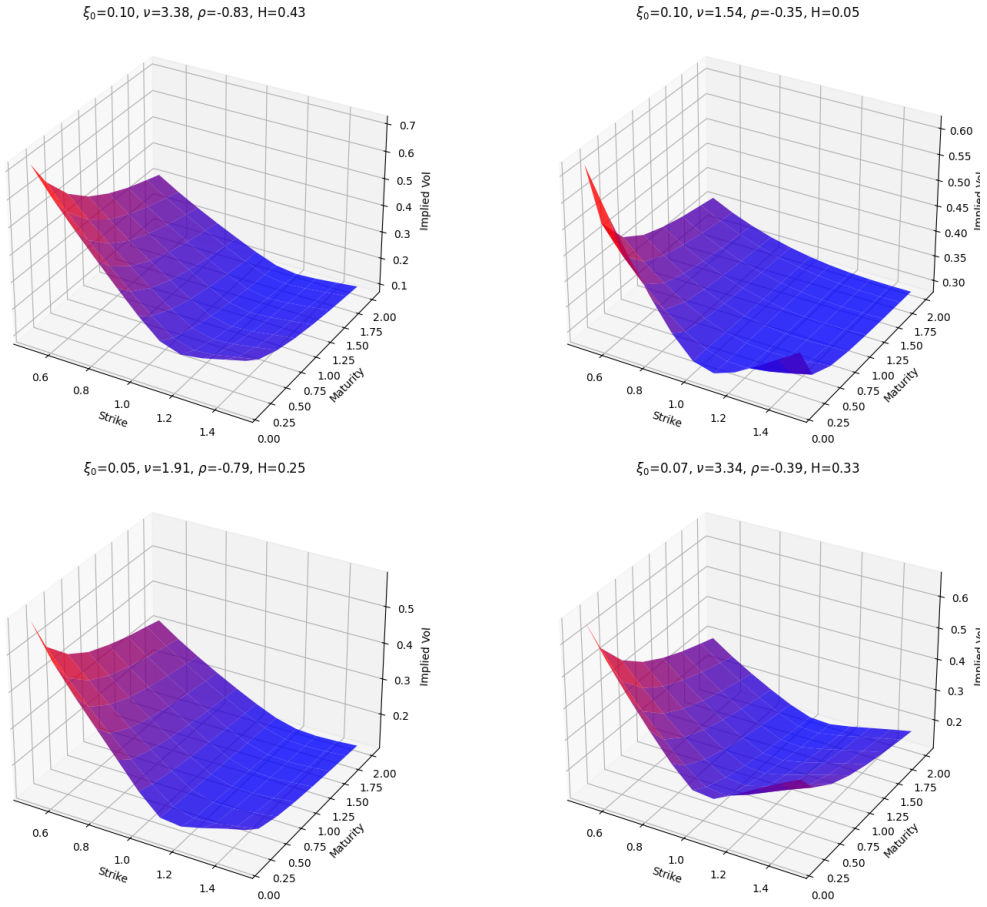

Figure 2: Volatility surfaces

4.2.2 Barrier

In parallel to the European call, we price a down-and-out (DO) digital barrier with lower barrier $\mathcal{B}$. Between grid points the discretised path may cross the barrier even when

both endpoints lie above it; Glasserman [5] corrects for this by weighting each path by the Brownian-bridge probability of *no* crossing. Over $[t_i, t_{i+1}]$ with local variance approximated by $V_{t_i}\,dt$ and endpoints $X(t_i), X(t_{i+1}) > \log \mathcal{B}$, this probability is

$$\hat{p}_i = 1 - \exp\left(-\frac{2\,(X(t_i) - \log \mathcal{B})\,(X(t_{i+1}) - \log \mathcal{B})}{V_{t_i}\,dt} \right),$$

and $\hat{p}_i = 0$ if either endpoint lies below $\log \mathcal{B}$. The down-and-out price is then the expected product of these survival probabilities, with the down-and-in following by in–out parity:

$$\mathrm{DO}(\theta) = \mathbb{E}\left[\prod_{i=0}^{n-1} \hat{p}_i \right], \qquad \mathrm{DI}(\theta) = 1 - \mathrm{DO}(\theta).$$

4.3 The Monte Carlo error floor

The surfaces used to label the training set are Monte Carlo estimates, so each entry carries sampling noise. This noise imposes an *error floor* on the entire procedure as the network cannot be more accurate than the data from which it learns. This is precisely the target $\widetilde{F} = \widetilde{P} + \mathcal{O}(\epsilon)$ introduced in Section 3. We quantify the floor by the relative half width of the 95% Monte Carlo confidence interval on each implied volatility, reported across the grid in the same format as the network error of Section 4.5 so that the two are directly comparable.

Standard error in volatility units. Each Monte Carlo call price C is a path average and carries a standard error SE_C. Since the network is trained on implied volatilities rather than prices, this error must be transported to the volatility scale. We invert both endpoints of the 95% price interval $[\,C - 1.96\,\mathrm{SE}_C,\ C + 1.96\,\mathrm{SE}_C\,]$ through the Black–Scholes map of Section 3 and take half of the resulting spread,

$$\mathrm{SE}_\sigma = \frac{\sigma_{\mathrm{BS}}(C + 1.96\,\mathrm{SE}_C) - \sigma_{\mathrm{BS}}(C - 1.96\,\mathrm{SE}_C)}{2 \times 1.96}.$$

The floor at a grid point (T, k) is the corresponding relative half width,

$$\mathrm{floor}_{T,k} = \frac{1.96\,\mathrm{SE}_\sigma}{\sigma_{\mathrm{BS}}(T, k)}.$$

Averaged over the dataset the floor is 1.23%, with a median of 0.57%. Figure 3 displays its mean and standard deviation across the grid.

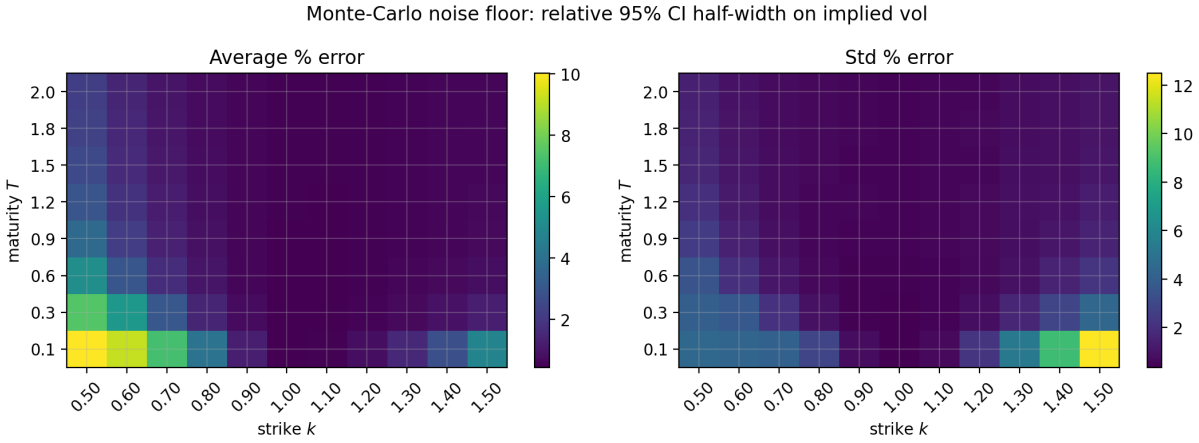

Figure 3: Monte Carlo error floor: mean and standard deviation of the relative 95% confidence interval half width on the implied volatility, across the 8×11 strike and maturity grid.

The floor is largest in the wings, at short maturities and extreme strikes, where the option has little time value and hence small vega, so price noise is magnified once expressed in volatility units, $\mathrm{SE}_\sigma \approx \mathrm{SE}_C/\mathrm{vega}$.

4.4 Neural-network training

4.4.1 Train/Test split

We split the 80,000 parameter–surface pairs once: 85% (68,000 pairs) form the training set and the remaining 15% (12,000 pairs) a held-out test set. Each pair has a parameter vector in $\mathbb{R}^4$ as feature and a surface in $\mathbb{R}^{88}$ as label.

The training set fixes the network weights through the empirical-risk minimisation of Section 3,

$$\widehat{w} = \operatorname*{argmin}_w \widehat{R}_N(F(\cdot, w)), \qquad \widehat{R}_N(F) = \frac{1}{N}\sum_{j=1}^{N} \left\| F(\theta^{(j)}) - \sigma^{(j)} \right\|_2^2.$$

The test set fixes how we measure it. Because the same data both fit and score the weights, the training risk $\widehat{R}_N$ underestimates the true generalisation risk $R(F) = \mathbb{E}\|F(\theta) - \sigma\|_2^2$. The held-out test risk $\widehat{R}_{\text{test}}$ does not share this bias, so we monitor it throughout training and report final performance on it (Section 4.5).

4.4.2 Normalisation

Both features and labels are normalised before training. Each parameter $\theta = (\xi_0, \nu, \rho, H)$ is mapped component-wise to $[-1, 1]$ by the affine transform $\theta \mapsto \big(2\theta - (\theta_{\min} + \theta_{\max})\big)/(\theta_{\max} - \theta_{\min})$, using the box bounds of Section 4.2. The surface labels are standardised to zero

mean and unit variance with `StandardScaler`, fit on the training labels alone and applied to both sets; predictions are returned to volatility units by inverting the transform.

4.4.3 Network architecture

The network is the class $\mathcal{NN}_5(4, 30, 30, 30, 30, 88, \text{ELU}, \text{ELU}, \text{ELU}, \text{ELU}, \text{id})$: an input of dimension 4, four hidden layers of 30 units with ELU activation, and a linear output layer of dimension 88. Algorithm 1 gives the forward pass.

Algorithm 1 Feed-forward neural network architecture

Input: Parameter vector $\theta = (\xi_0, \nu, \rho, H) \in \mathbb{R}^4$
Output: Predicted implied volatility surface $\widetilde{F}(\theta) \in \mathbb{R}^{88}$
Define the input layer with dimension 4
$x_1 \leftarrow \text{ELU}(W_1\theta + b_1), \quad x_1 \in \mathbb{R}^{30}$
$x_2 \leftarrow \text{ELU}(W_2x_1 + b_2), \quad x_2 \in \mathbb{R}^{30}$
$x_3 \leftarrow \text{ELU}(W_3x_2 + b_3), \quad x_3 \in \mathbb{R}^{30}$
$x_4 \leftarrow \text{ELU}(W_4x_3 + b_4), \quad x_4 \in \mathbb{R}^{30}$
$\widetilde{F}(\theta) \leftarrow W_5x_4 + b_5, \quad \widetilde{F}(\theta) \in \mathbb{R}^{88}$
Compile the model using the mean squared error loss
Optimise the network parameters using the Adam optimiser

for a total of $(4 \times 30 + 30) + 3 \times (30 \times 30 + 30) + (30 \times 88 + 88) = 5668$ weights and biases. Two of these choices are deliberate.

Activation. We use ELU rather than ReLU because $\sigma_{\text{ELU}} \in C^1(\mathbb{R})$. By the universal approximation theorem for derivatives of Section 2.1, this lets the network approximate not only F^* but also its gradient, $\nabla_\theta \widetilde{F} \approx \nabla_\theta F^*$, the smoothness on which the gradient-based calibration of Section 5 relies. A non-differentiable choice such as ReLU would not provide it.

Depth. The four hidden layers follow the original architecture [1], beyond which no material gain is observed for this map.

Figure 4 shows the resulting architecture.

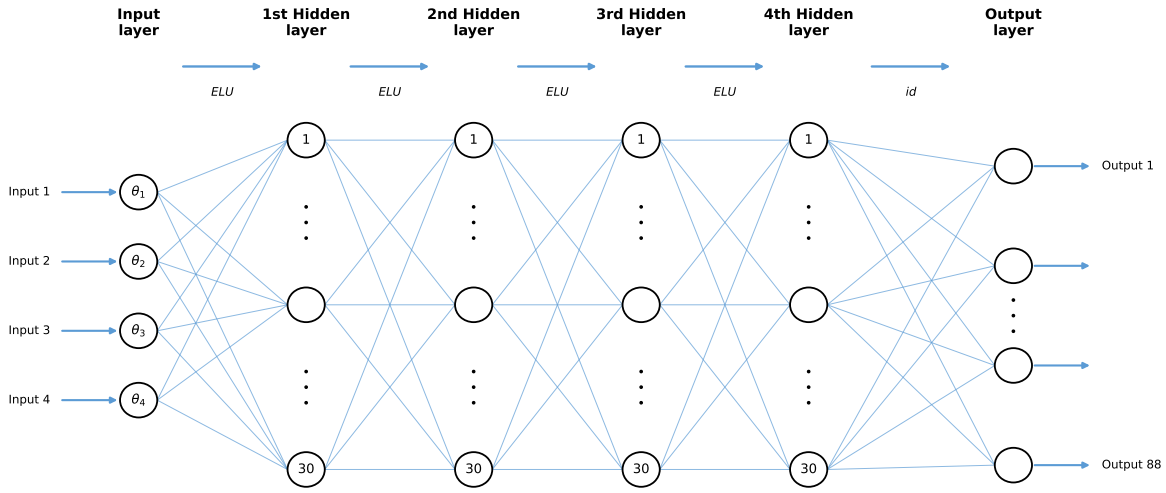

Figure 4: Neural Network structure

4.4.4 Training

We minimise the empirical risk $\widehat{R}_N$ over the weights w with the Adam optimiser, a mini-batch stochastic gradient descent scheme, using batch size 32 and a budget of 200 epochs. After each epoch we record both the training risk and the test risk $\widehat{R}_{\text{test}}$, the unbiased estimator of the generalisation risk from Section 4.4.

Early stopping regularises the fit: training halts once $\widehat{R}_{\text{test}}$ has not improved for 25 consecutive epochs, and the weights are restored to those of the best epoch. This guards against overfitting, the regime in which the training risk keeps falling while the test risk has flattened.

Figure 5 shows the two risks over training. They descend together and plateau jointly, with no widening gap, indicating that the network generalises rather than memorises and that the stopping point is well chosen.

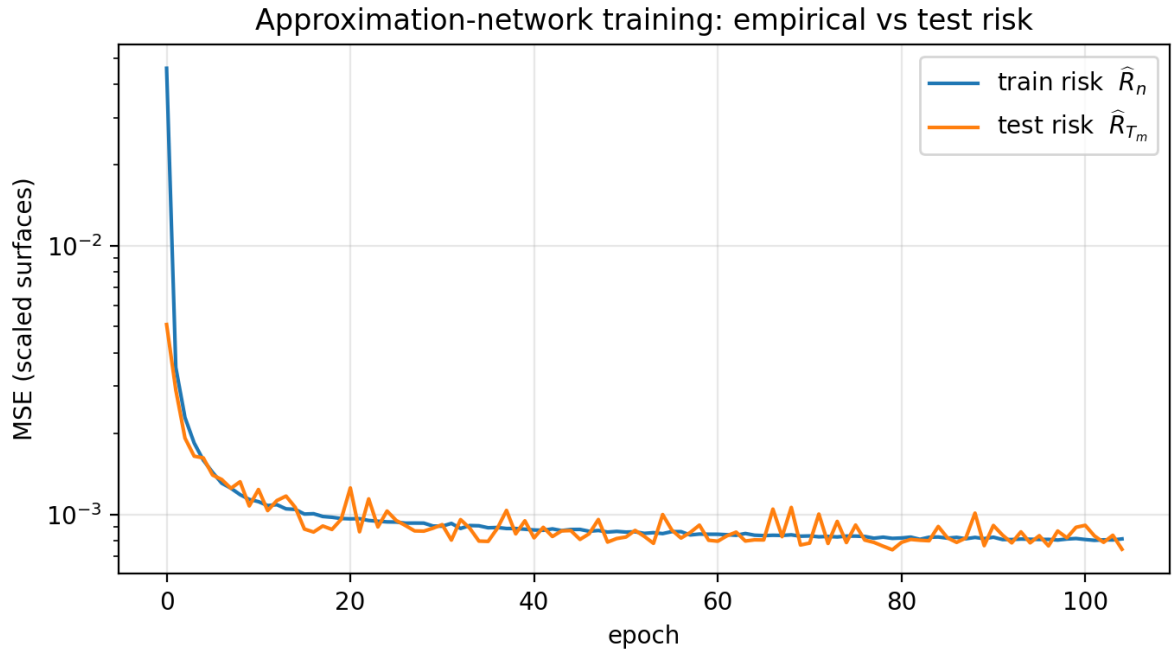

Figure 5: Training and test risk over epochs (MSE on the scaled surfaces, log scale).

4.5 Results

With the weights frozen, we evaluate the trained map on the 12000 held-out test surfaces and compare its outputs against the Monte Carlo labels. Figure 6 reports the relative implied-volatility error per grid point, in the same two-panel format (mean and standard deviation) as the Monte Carlo floor of Figure 3 so that the two are read side by side, while Figure 7 shows the fitted smiles for the best- and worst-fit test surfaces.

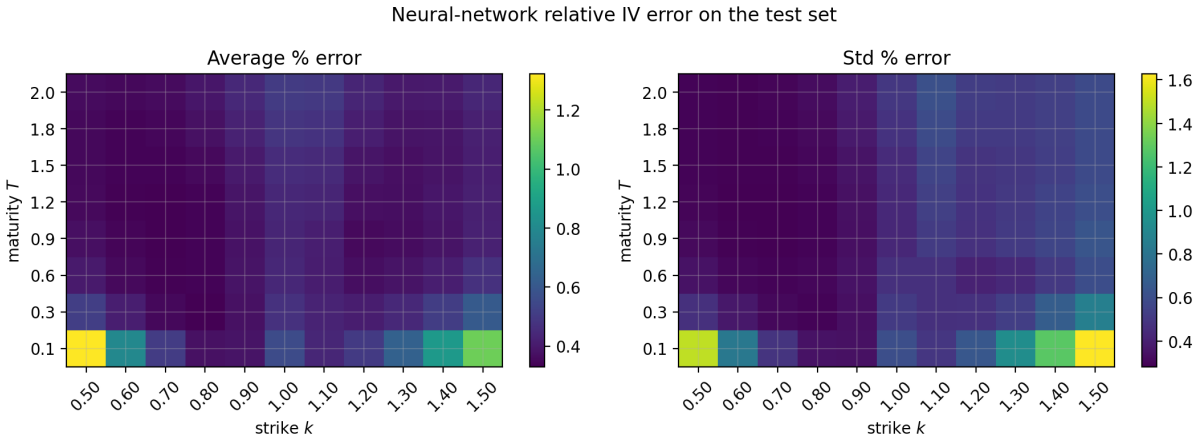

Figure 6: Neural-network relative implied-volatility error on the test set: mean and standard deviation across the 8×11 grid.

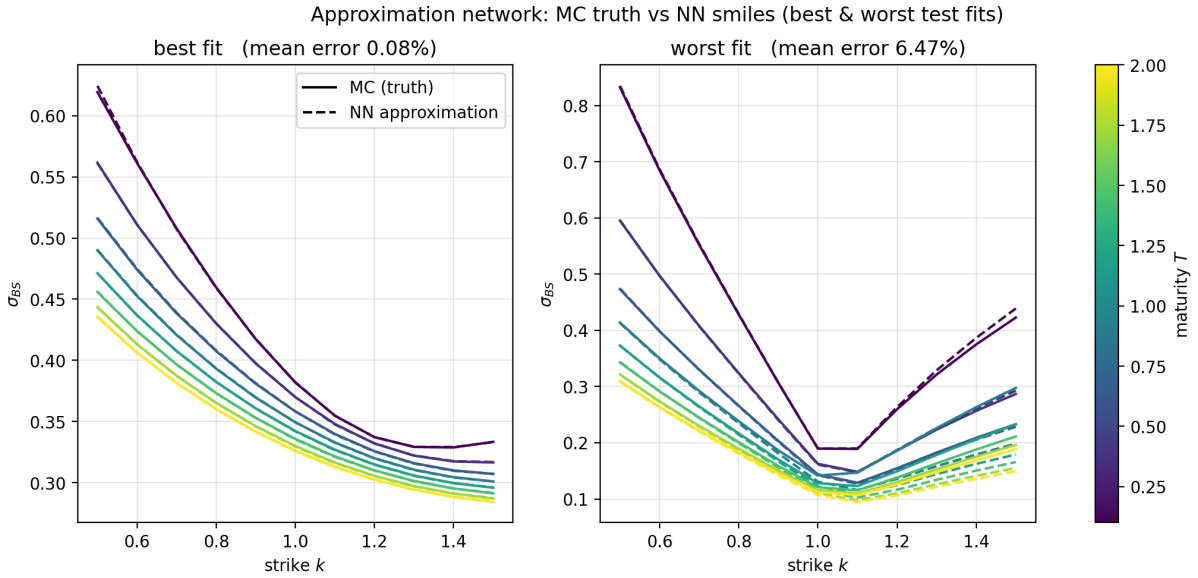

Figure 7: Monte Carlo prices against the neural-network approximation for the best- and worst-fit test surfaces.

The network attains a mean relative implied-volatility error of 0.43% on the test set (Figure 6). This is small in practical terms: it lies below a typical bid-ask spread, reported by Horvath et al. [1] as about 0.2% in volatility terms, roughly 1% in relative terms, for liquid options under one year. The approximation is therefore well within the tolerance needed to calibrate to market data. The error is also of the same order as the Monte Carlo sampling floor of 1.23% (Section 4.3), so the network is about as accurate as the data it was trained on. The largest residuals occur at short maturities and at the extreme strikes, where the options carry little time value and the implied volatility is hardest to pin down. Figure 7 shows that even the worst-fit test surface tracks the Monte Carlo smiles closely.

5 Online calibration

This section realises step (ii) of the framework of Section 3. Section 4 produced a trained network $\widetilde{F}$ that returns an implied-volatility surface for any parameter vector at negligible cost. We now use it to calibrate the model. Calibration is the approximate problem (4) with the Monte Carlo pricer $\widetilde{P}$ replaced by $\widetilde{F}$. Because $\widetilde{F}$ is deterministic and differentiable, the optimisation that was intractable under Monte Carlo reduces to a routine least-squares fit. We describe the method (Section 5.1), calibrate the network to held-out surfaces with known parameters to measure its accuracy and speed (Section 5.2), and compare optimisers to justify our choice (Section 5.3).

5.1 Method

5.1.1 Calibration as least squares

Given a target surface $\Sigma_{BS}^{\mathrm{MKT}}$, calibration solves

$$\hat{\theta} = \underset{\theta \in \Theta}{\operatorname{argmin}} \left\| \widetilde{F}(\theta) - \Sigma_{BS}^{\mathrm{MKT}} \right\|_2^2.$$

We solve it in the scaled coordinates of Section 4.4, with the parameters mapped to $[-1, 1]$ and the surfaces standardised, so that every residual component is on a comparable scale. The optimiser starts from the centre of the parameter box and the recovered vector is returned to model units afterwards. The objective is a sum of squares over the 88 grid points, the natural form for a least-squares solver.

5.1.2 An exact gradient from the frozen network

The fit is gradient-based and therefore requires the Jacobian $\nabla_\theta \widetilde{F}$. Since the network is an explicit composition of affine maps and the C^1 ELU activation, this Jacobian is available in closed form and is evaluated exactly by the chain rule, with no finite differencing. This is the practical payoff of the activation chosen in Section 2.1: the derivative-approximation guarantee $\nabla_\theta \widetilde{F} \approx \nabla_\theta F^*$ holds, so the optimiser follows a faithful gradient of the true pricing map. A ReLU network, not being C^1, would not provide this.

For speed inside the optimiser loop we evaluate the forward pass and the analytic Jacobian with a lightweight `NumPy` copy of the trained weights. This copy reproduces the Keras network to machine precision, with a maximum absolute difference of 8.9×10^{-16} on a test input. By contrast, a brute-force Monte Carlo calibration has no closed-form gradient and must approximate it by finite differences, which is both slower and noisier.

5.1.3 The Levenberg–Marquardt optimiser

We carry out the fit with the Levenberg–Marquardt algorithm, which is designed for least-squares objectives and uses the residual and its Jacobian directly. The reasons for preferring it to the alternatives are given in Section 5.3.

5.2 Results

We assess the calibration on the held-out test surfaces of Section 4.5. Each test surface was generated from a known parameter vector, so recovering that vector provides a controlled, self-consistent measure of calibration quality.

The procedure is fast. Calibrating 1000 test surfaces takes 0.37 ms each on average. Measured against the Monte Carlo pricing cost of Section 4.1, this is the order-of-magnitude speed-up that the two-step design is built to deliver, consistent with the speed-ups reported in the original paper.

We report accuracy with two measures: the relative error $|\hat{\theta} - \theta|/|\theta|$ of each recovered parameter, and the root-mean-square error (RMSE) between the target surface and the surface repriced at $\hat{\theta}$. The median parameter relative errors are 0.59% for ξ_0, 0.65% for ν, 0.91% for ρ and 1.76% for H. The surface RMSE has a median of 0.095% and a 99% quantile of 0.438%, inside the benchmark 99% quantile of 1% used in the paper. Figure 8 shows the empirical distributions of both quantities across the test set.

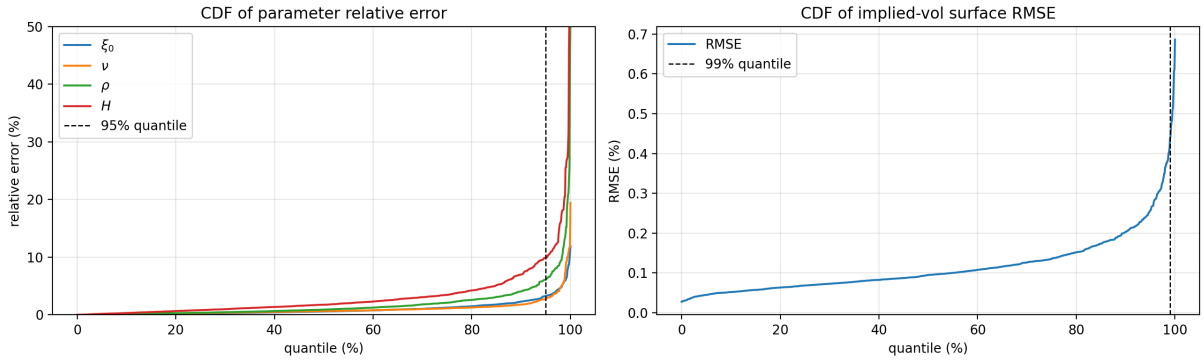

Figure 8: Empirical cumulative distribution functions of the parameter relative error (left, with the 95% quantile marked) and of the implied-volatility surface RMSE (right, with the 99% quantile marked), across the calibrated test surfaces.

The parameter error is not uniform across the parameter space. Figure 9 resolves it against the true value, one panel per parameter. The Hurst parameter H, and to a lesser extent the correlation ρ, are the least identifiable, with the largest errors at small H and at ξ_0 near its lower bound. In those regions the surface is least sensitive to the parameter, so a range of values reproduces it almost equally well, and the small denominator further inflates the relative error. This reflects the conditioning of the calibration problem rather than a deficiency of the network.

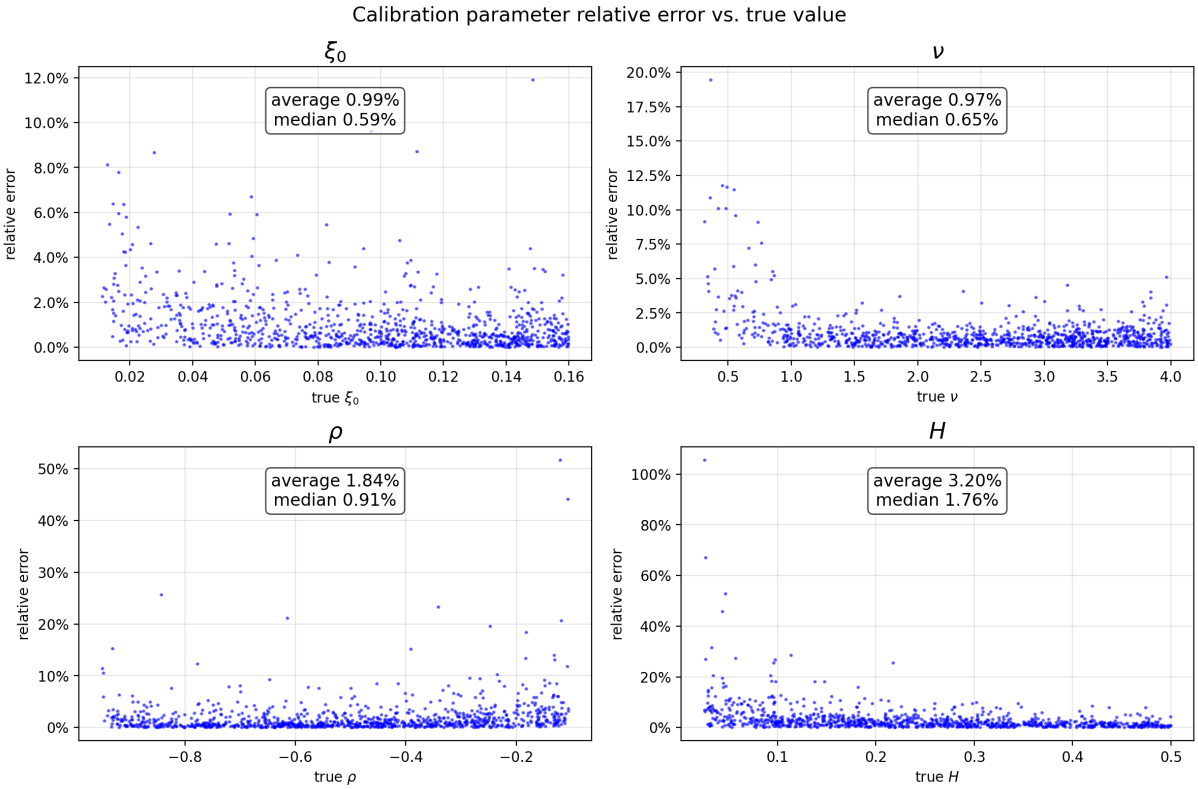

Figure 9: Calibration parameter relative error against the true parameter value, one panel per parameter. The error is largest for small H and for ξ_0 near its lower bound, where the surface is least sensitive to the parameter.

Read as implied-volatility smiles, the fits are close even in the worst case. Figure 10 shows the best- and worst-fit test surfaces, ranked by mean relative error across the grid; the two mean errors are 0.06% and 1.89%. The calibrated smiles track the targets across maturities in both panels.

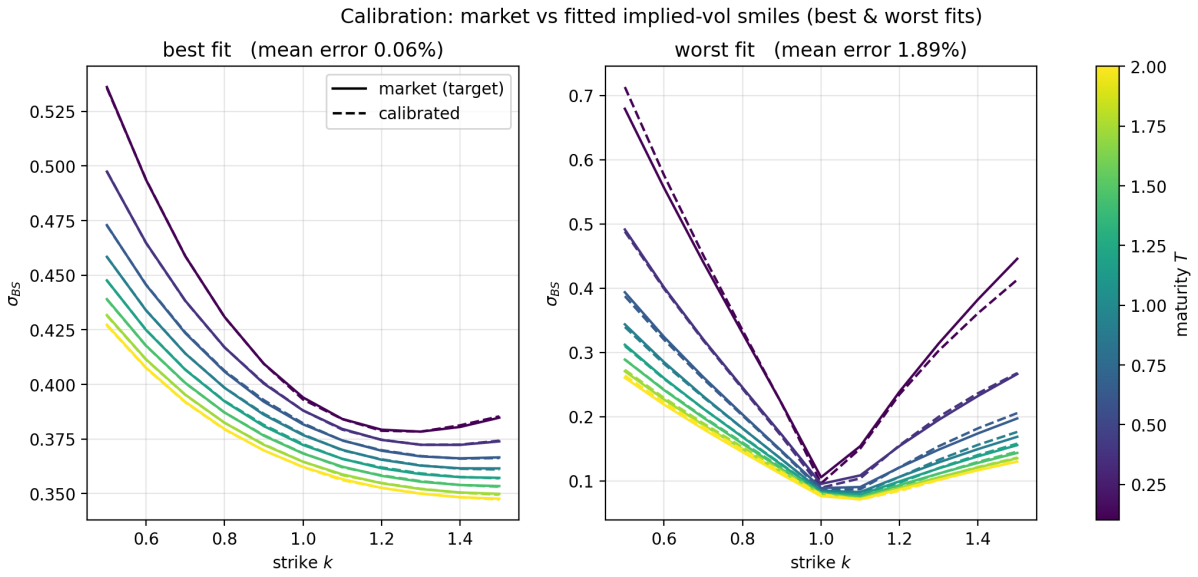

Figure 10: Market target against the calibrated surface, read as implied-volatility smiles, for the best- and worst-fit test surfaces. Solid curves are the target, dashed curves the calibrated fit; colour denotes maturity.

5.3 Choosing the optimiser

Levenberg–Marquardt is one of several optimisers able to solve the frozen-network calibration. We compare seven, in two families (cf. Horvath et al. [1], Table 1). The gradient-based methods, Levenberg–Marquardt, BFGS, L-BFGS-B and SLSQP, use the exact Jacobian of Section 5.1 and converge in few iterations, but rely on the smoothness that the C^1 activation provides. The gradient-free methods, Nelder–Mead, COBYLA and Differential Evolution, require no derivatives and can search globally, at the cost of many more function evaluations.

Figure 11 reports the average calibration time per surface for each optimiser on a logarithmic scale. The gradient-based methods are one to three orders of magnitude faster than the gradient-free ones while reaching the same surface RMSE, around 0.10% in every case. Levenberg–Marquardt is the fastest of all, at 0.36 ms per surface, which is why we adopt it in Section 5.2. This reproduces the paper's finding.

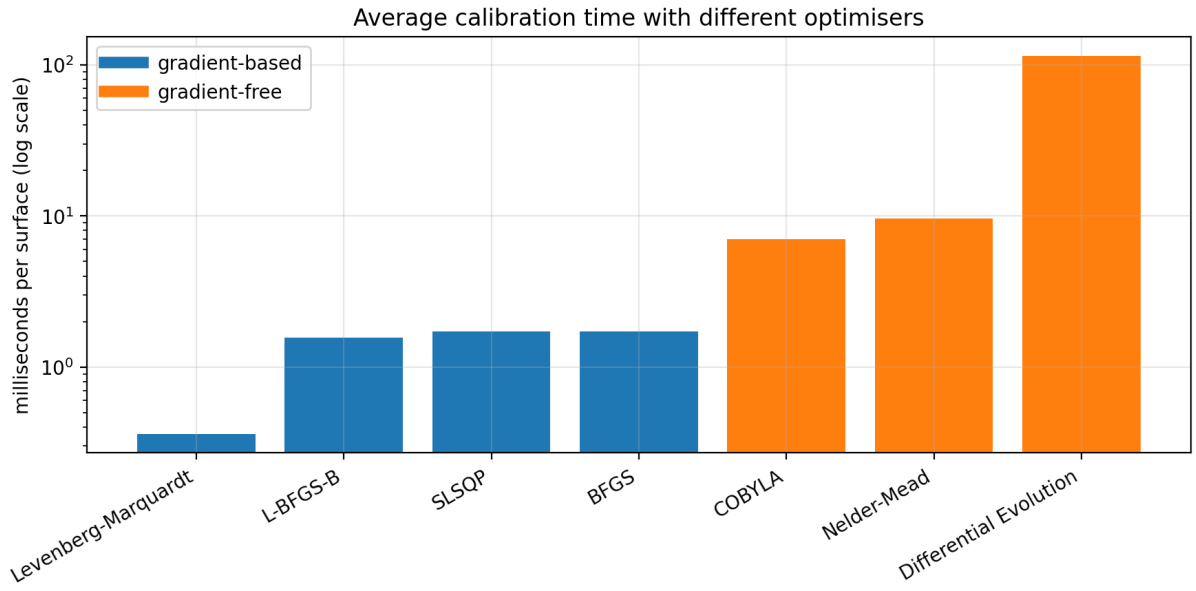

Figure 11: Average calibration time per surface for seven optimisers (log scale). Gradient-based methods (blue) are far faster than gradient-free ones (orange), with Levenberg–Marquardt the fastest.

5.4 Scope and extensions

We calibrated the rough Bergomi model with a flat forward-variance level to simulated surfaces. Two extensions follow the paper directly. The first is a piecewise-constant forward-variance curve $\xi_0(t)$, with one value per maturity, which adds eight inputs to the network and a realistic term structure. The second is calibration to historical market data: Horvath et al. [1] calibrate the rough Bergomi model to ten years of SPX implied-volatility surfaces and recover a Hurst parameter $H < 1/2$ throughout, in line with the empirical roughness of equity volatility.

References

[1] B. Horvath, A. Muguruza, M. Tomas, *Deep learning volatility: A deep neural network perspective on pricing and calibration in (rough) volatility models* (2021).

[2] B. Horvath, A. Jacquier, A. Muguruza, A. Søjmark, *Functional central limit theorems for rough volatility* (2024).

[3] J. Gatheral, T. Jaisson, M. Rosenbaum, *Volatility is rough* (2018).

[4] M. Bennedsen, A. Lunde, M. S. Pakkanen, *Hybrid scheme for Brownian semistationary processes* (2017).

[5] P. Glasserman, *Monte Carlo Methods in Financial Engineering* (2004).